

Website Fingerprinting Analysis

Computer Security

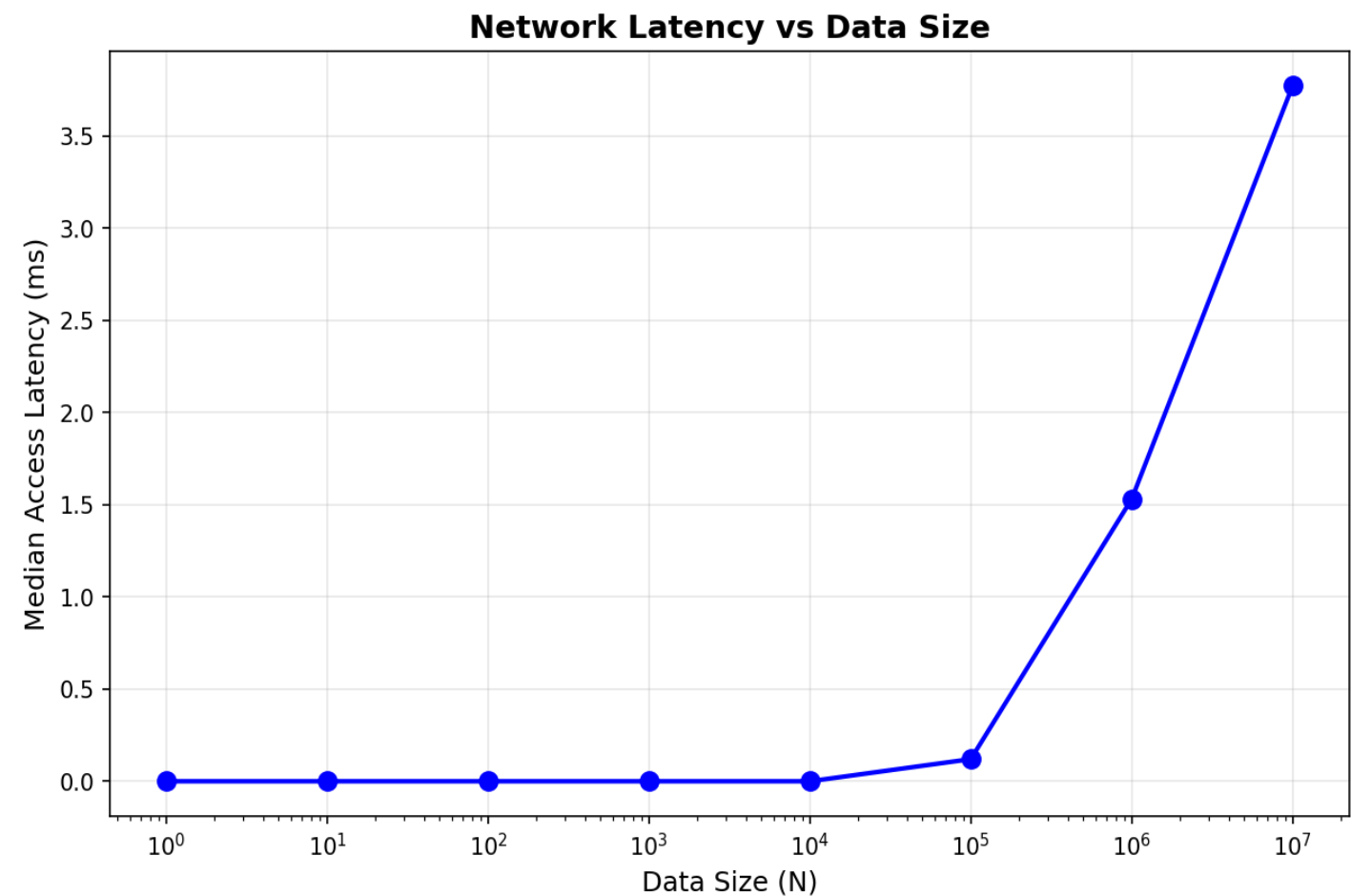
Assignment-02

Student ID: 2005021

Project Overview

- Objective
 - • Develop and compare deep learning models for website fingerprinting
 - • Analyze network traffic patterns to identify visited websites
 - • Evaluate model performance on real-world data
- Dataset
 - • 3 websites: BUET Moodle, Google, Prothom Alo
 - • 1000 traces per website (3000 total samples)
 - • Network latency measurements for different data sizes

Latency Analysis



Key Insights:

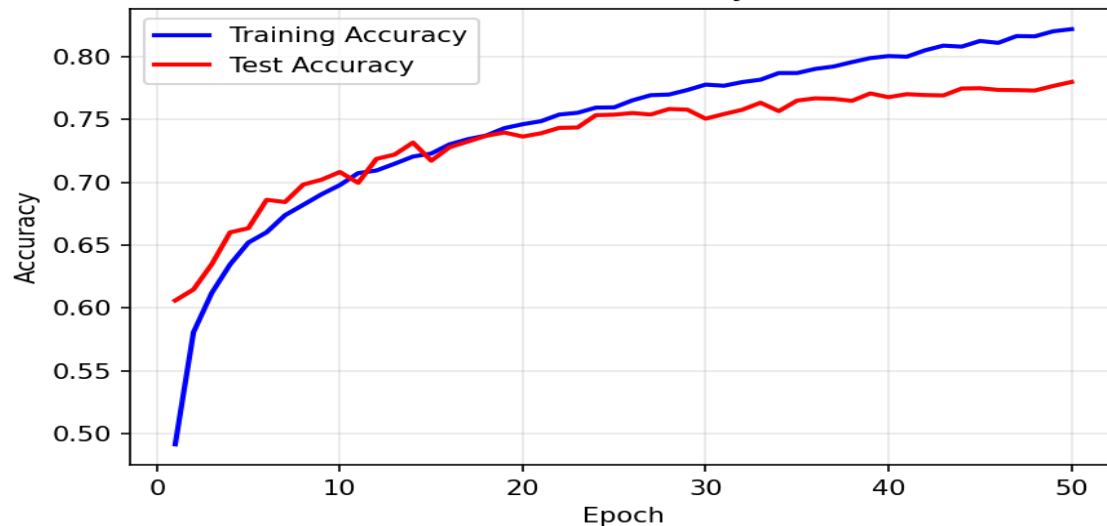
- Latency negligible for $N < 100K$
- Significant increase at 1M+ samples
- Logarithmic growth pattern

Model Architectures Tested

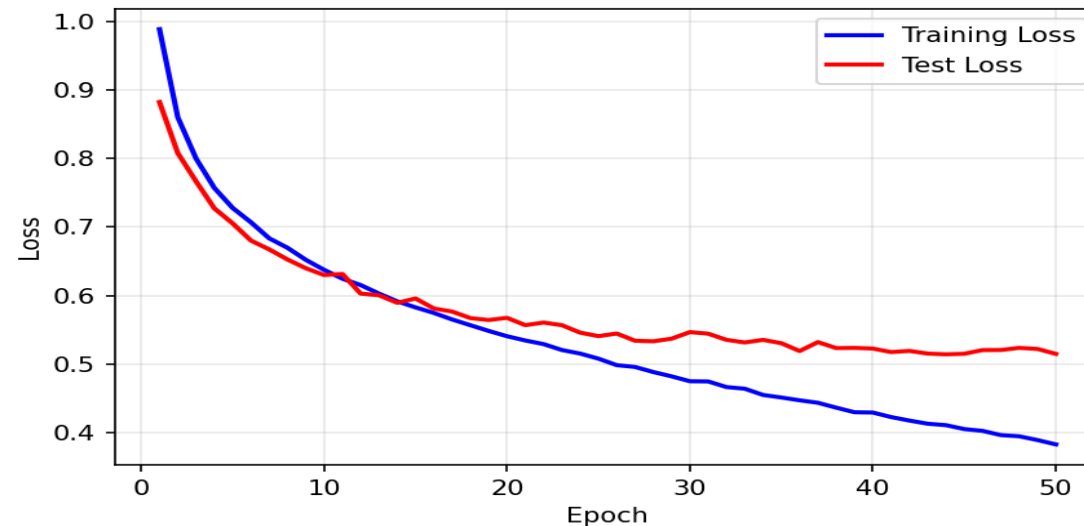
- 1. Basic CNN
 - • Simple convolutional neural network
 - • Baseline model for comparison
- 2. Complex CNN
 - • Deeper architecture with more layers
 - • Batch normalization and dropout
- 3. Transformer
 - • Attention-based architecture
 - • Self-attention mechanism for sequence modeling
- 4. CNN-LSTM & CNN-BiLSTM-Attention
 - • Hybrid architectures combining CNN and RNN
 - • Bidirectional LSTM with attention mechanism

Training Results: Basic CNN

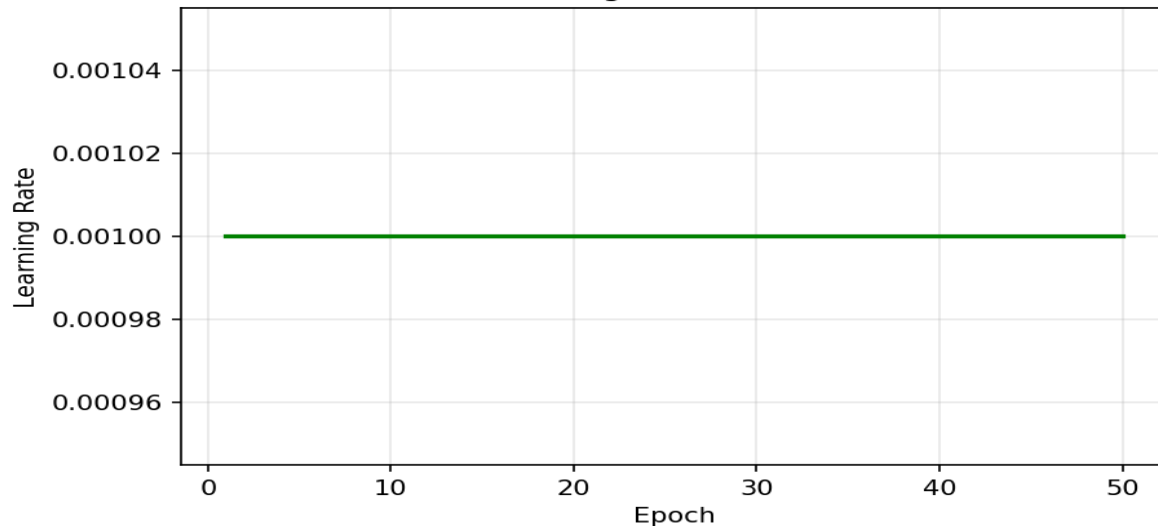
Model Accuracy



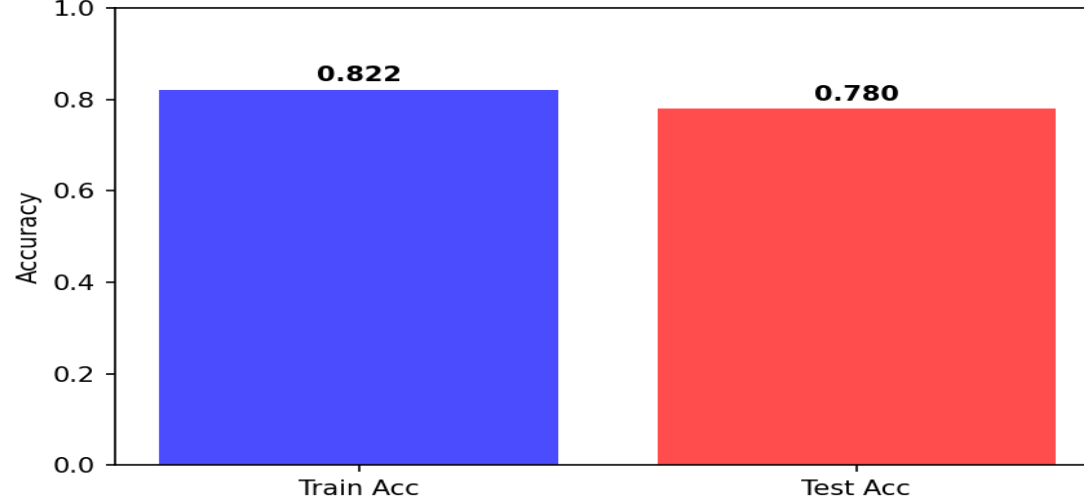
Model Loss



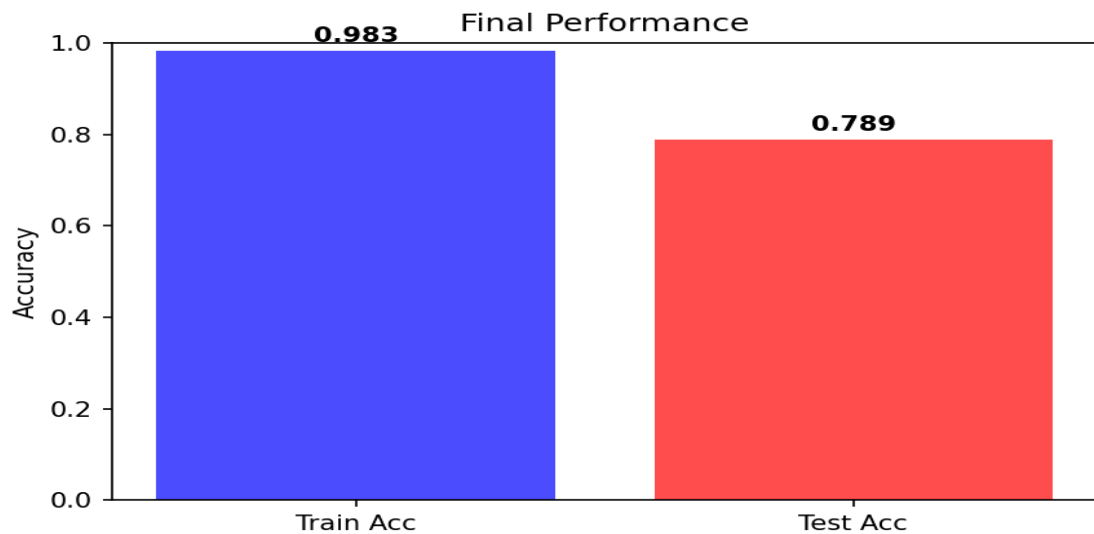
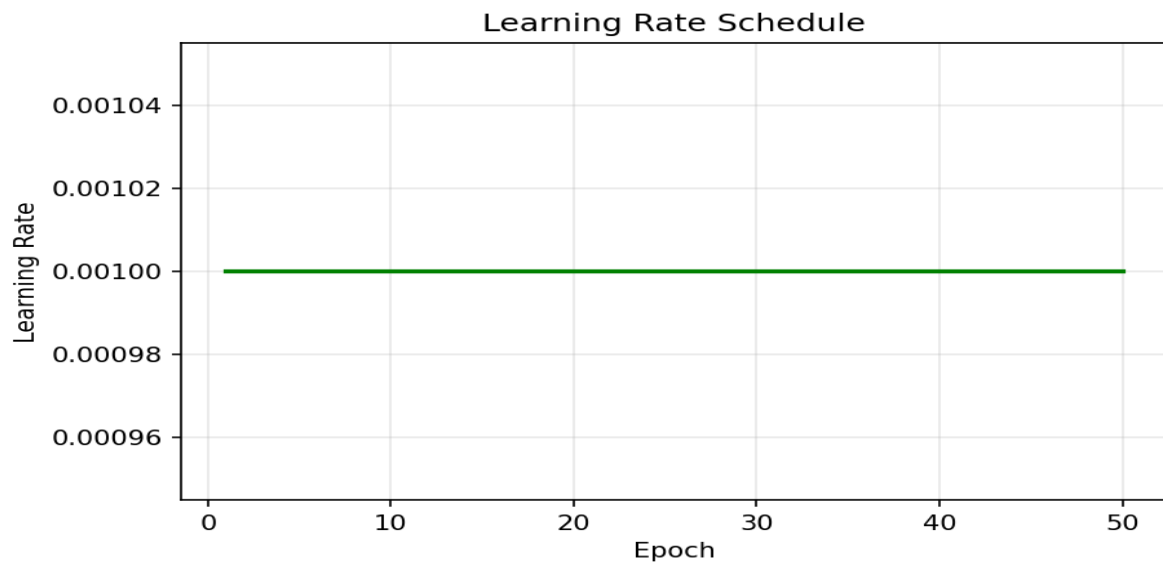
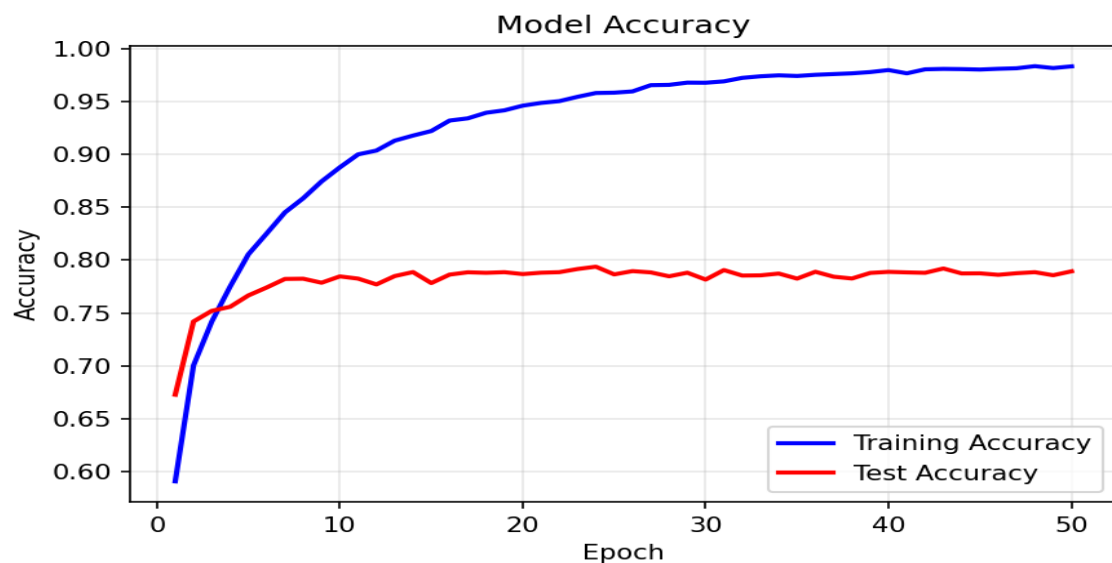
Learning Rate Schedule



Final Performance

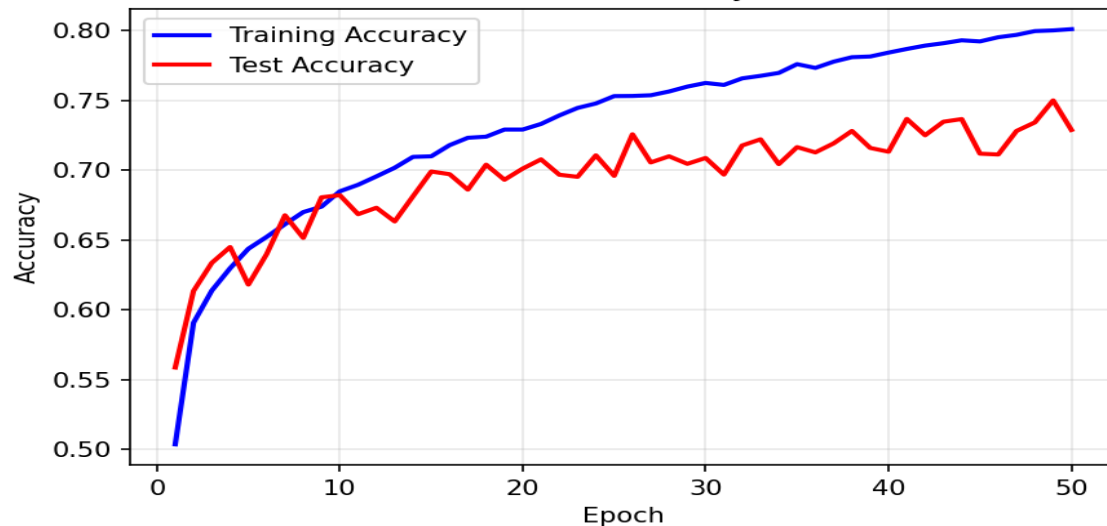


Training Results: Complex CNN

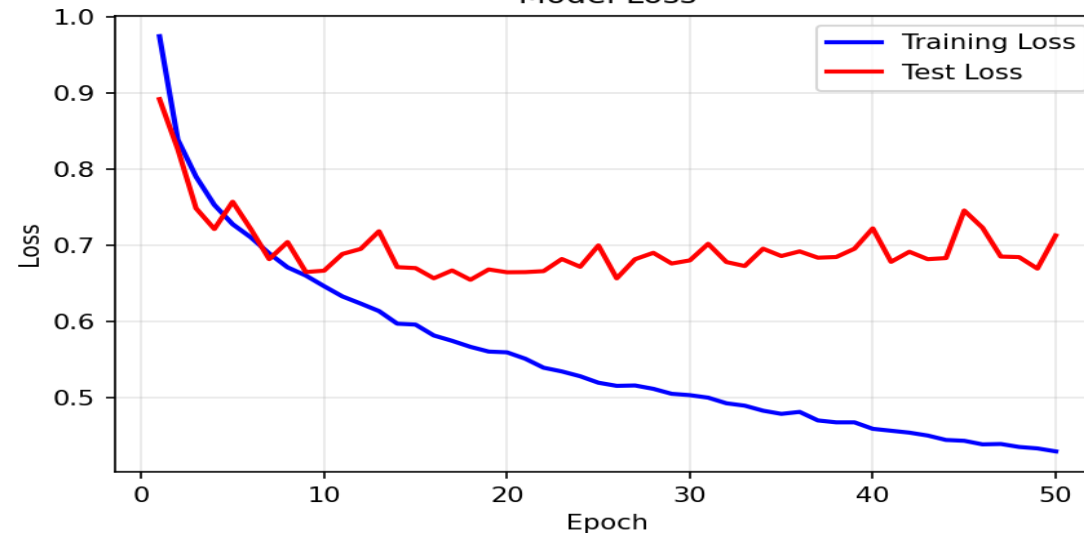


Training Results: Transformer

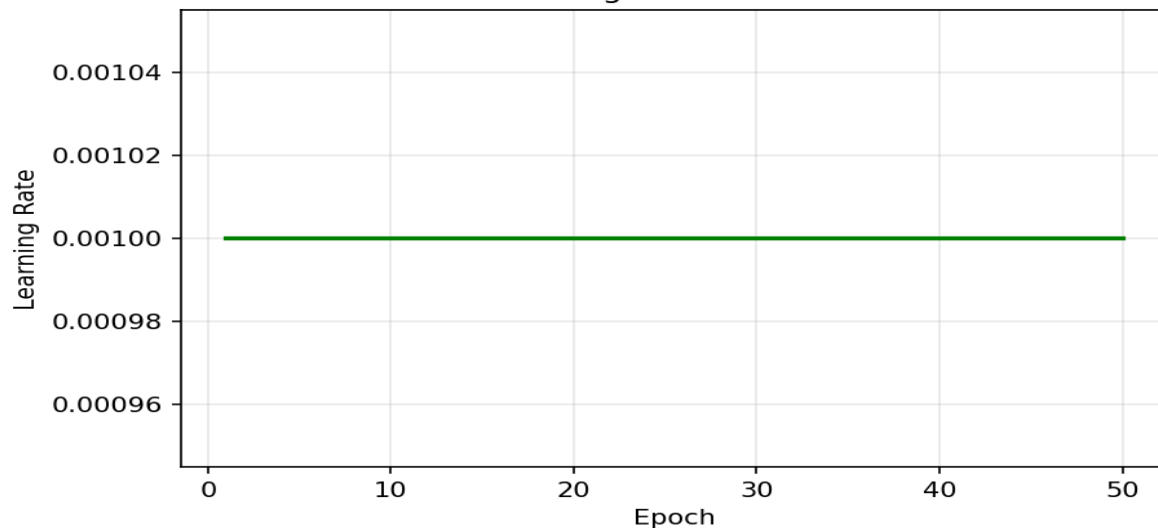
Model Accuracy



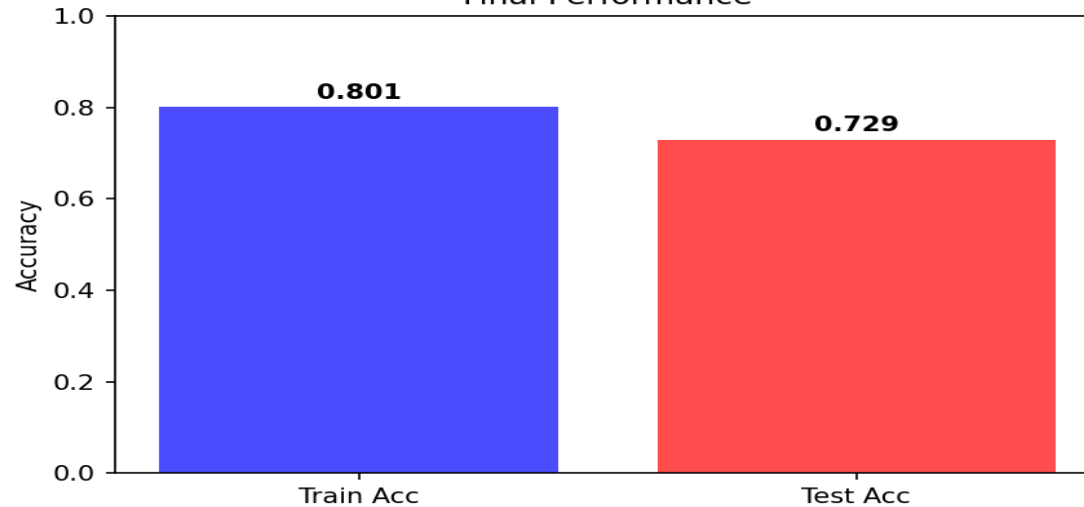
Model Loss



Learning Rate Schedule

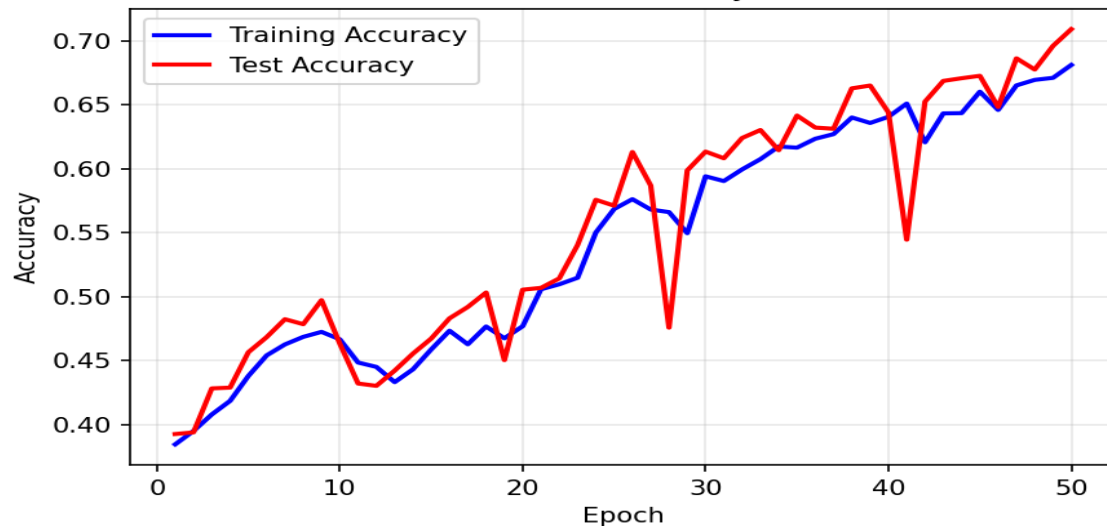


Final Performance

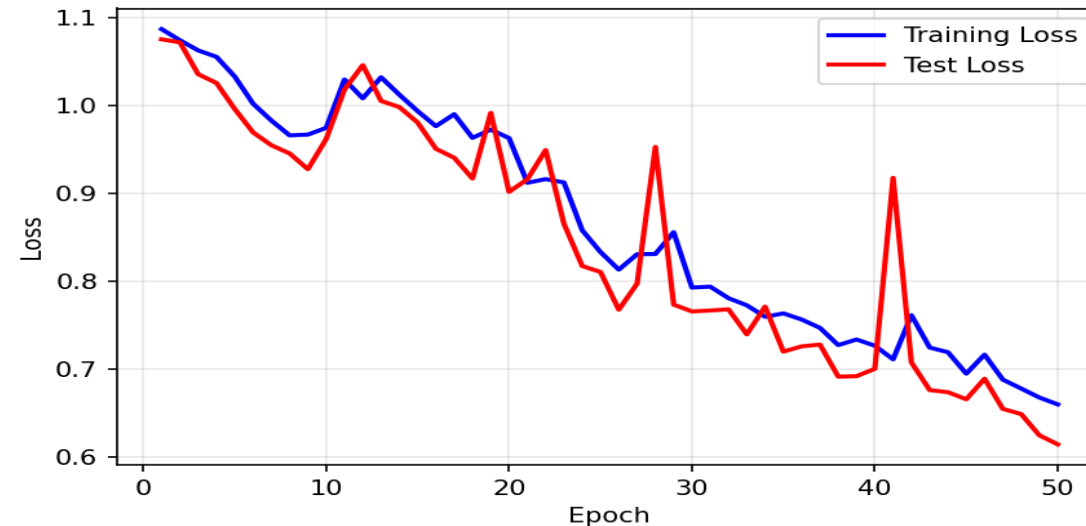


Training Results: CNN-LSTM

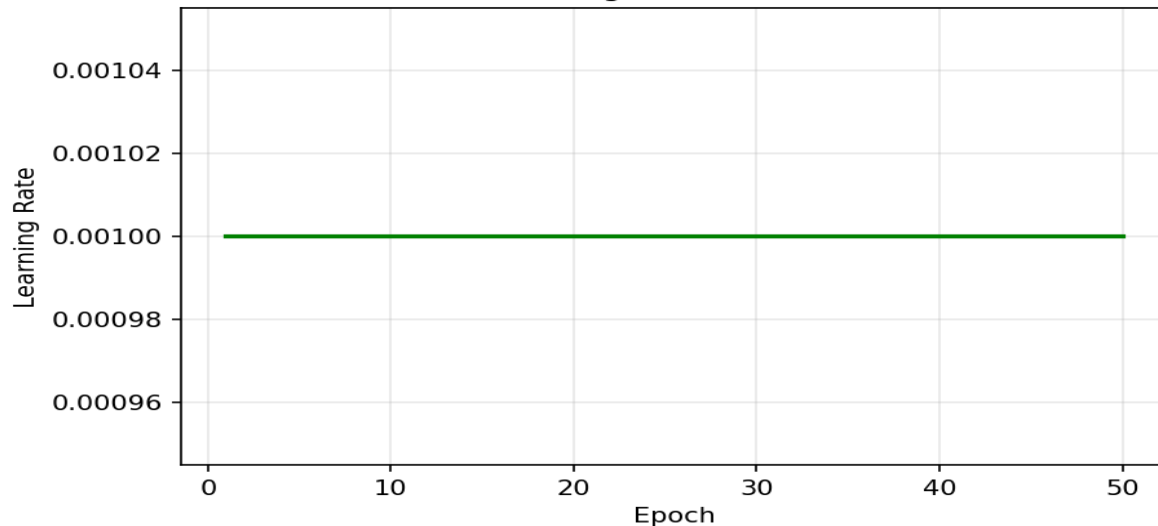
Model Accuracy



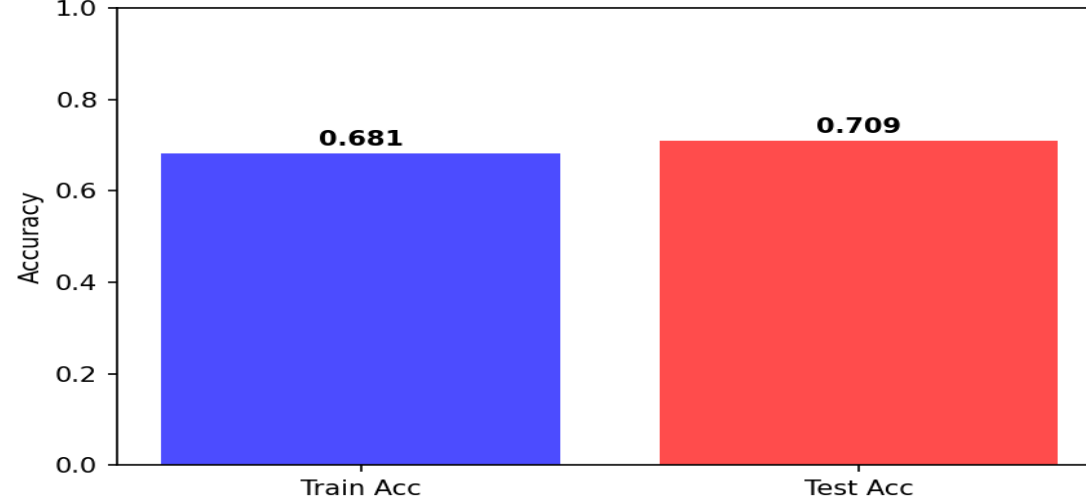
Model Loss



Learning Rate Schedule

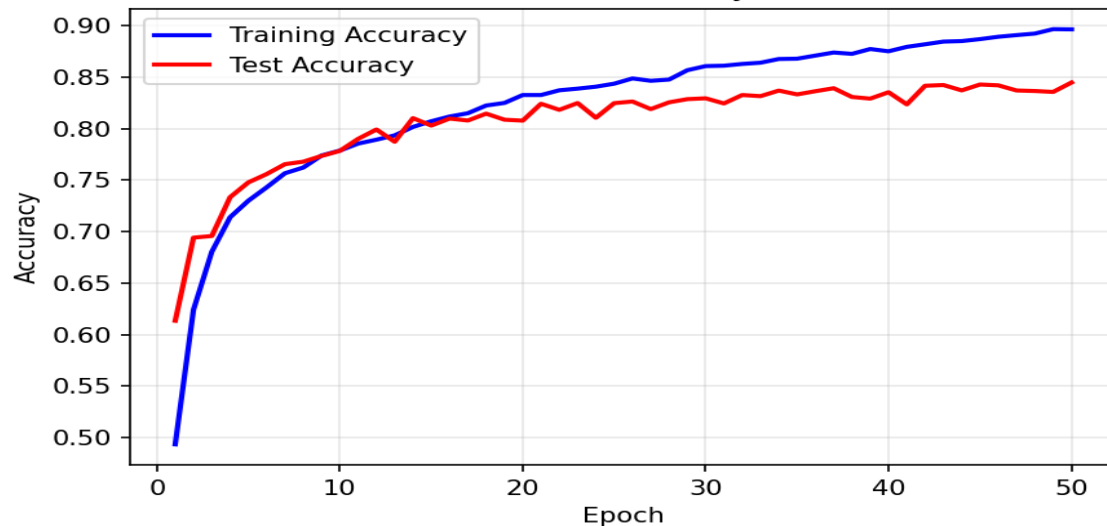


Final Performance

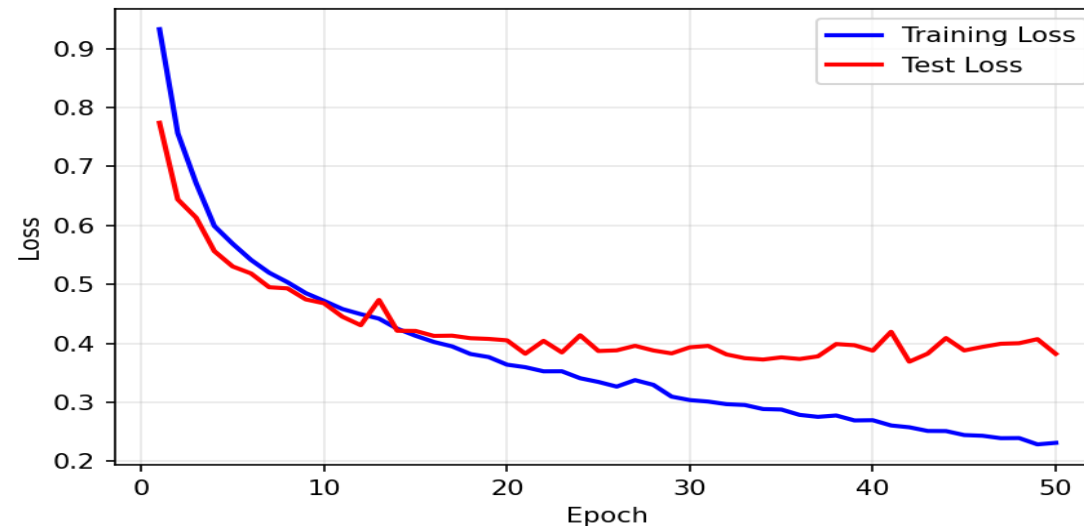


Training Results: CNN-BiLSTM-Attention

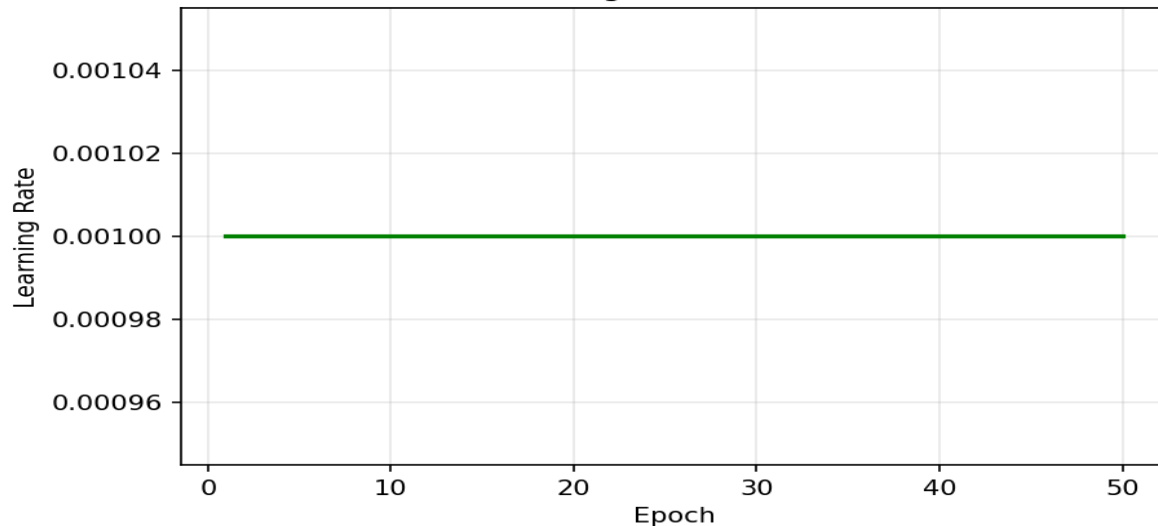
Model Accuracy



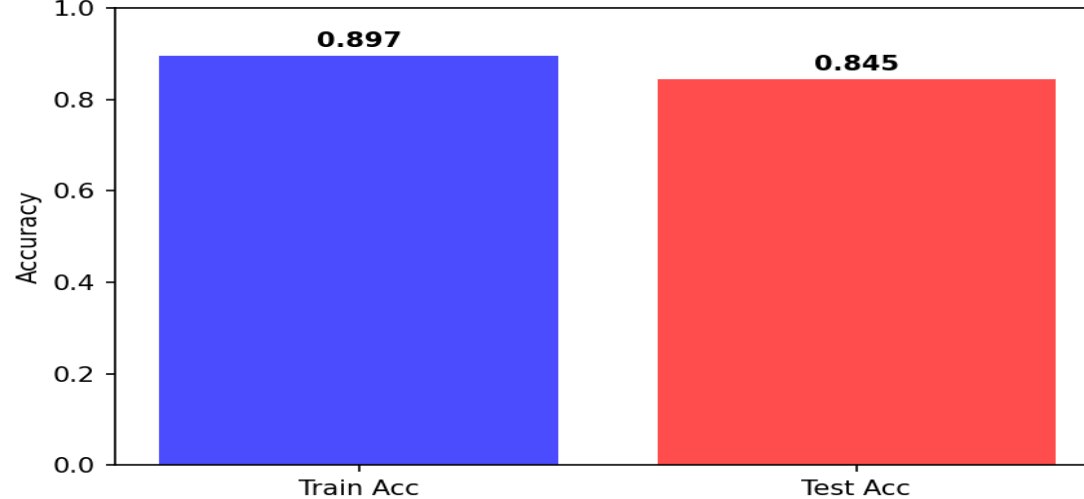
Model Loss



Learning Rate Schedule



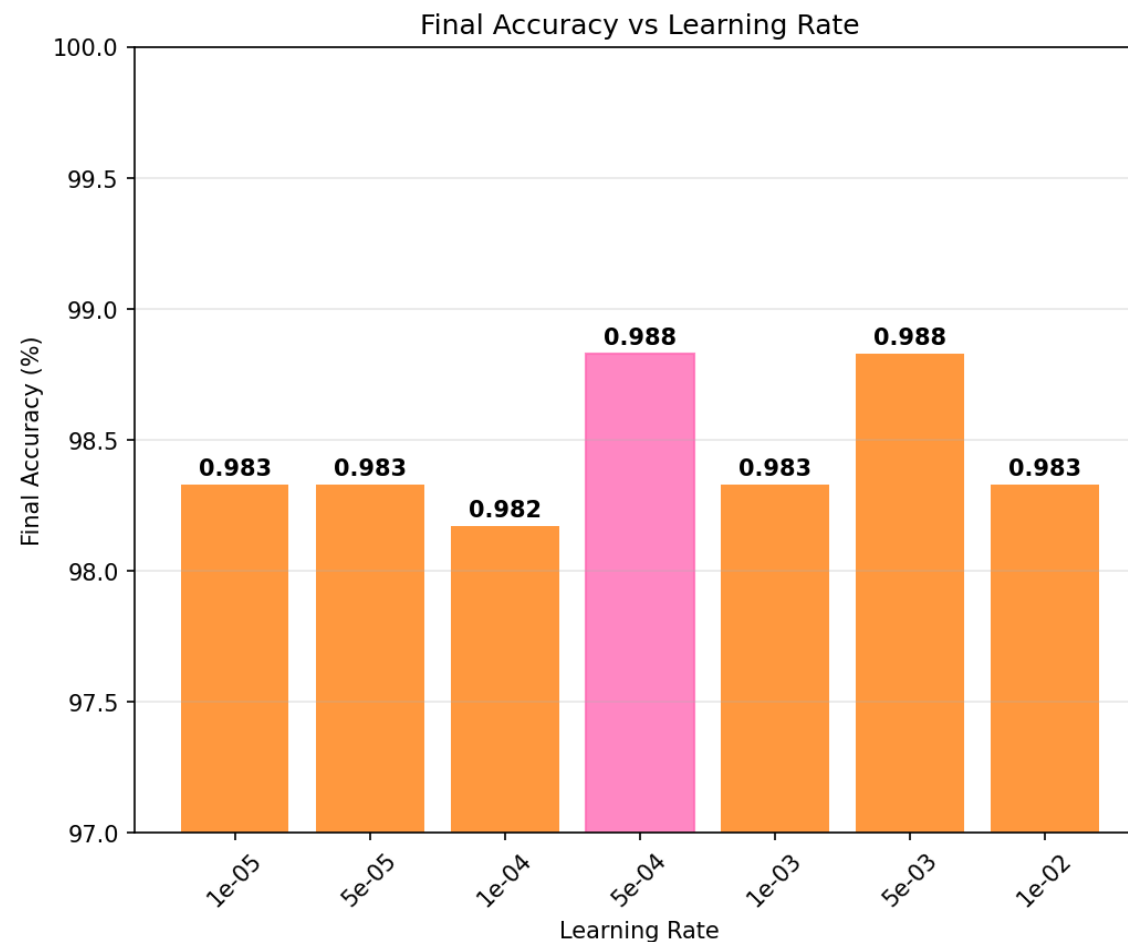
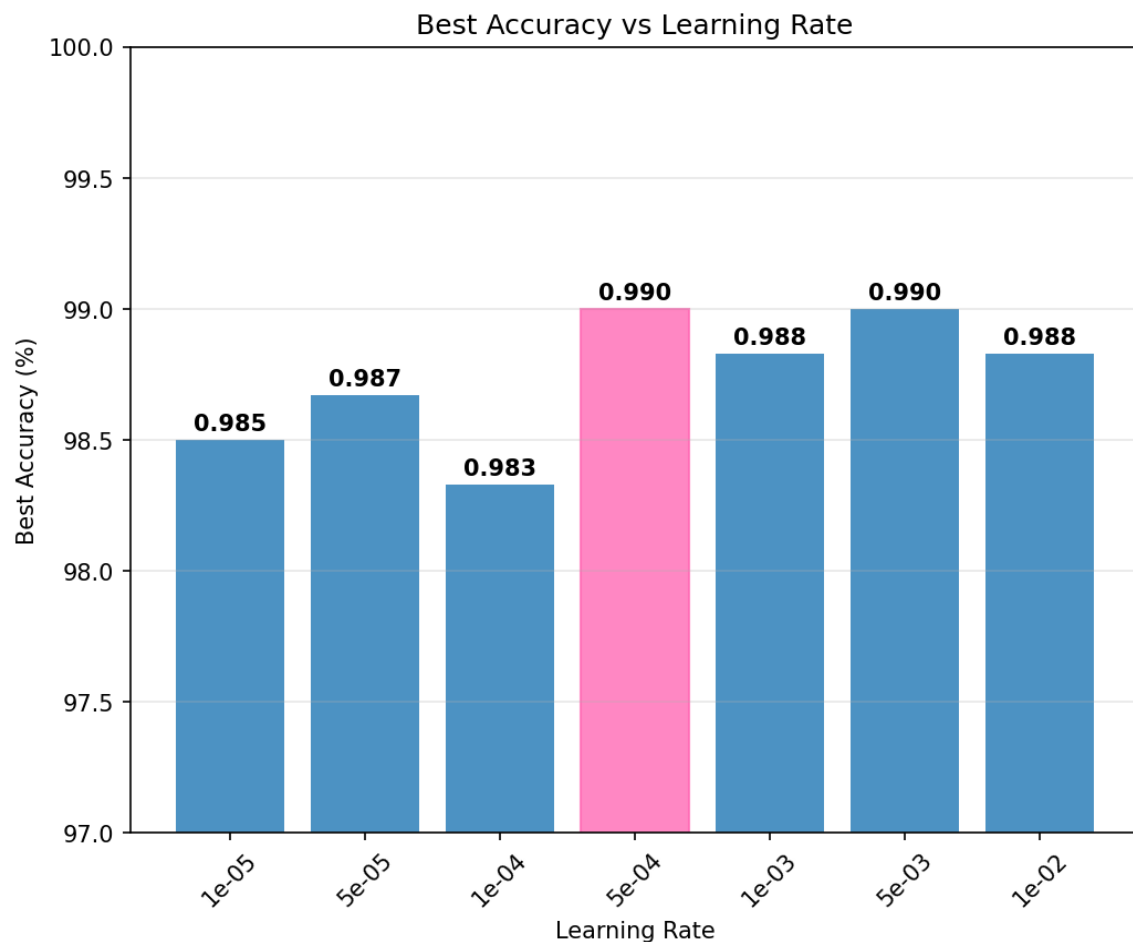
Final Performance



Hyperparameter Optimization Experiments

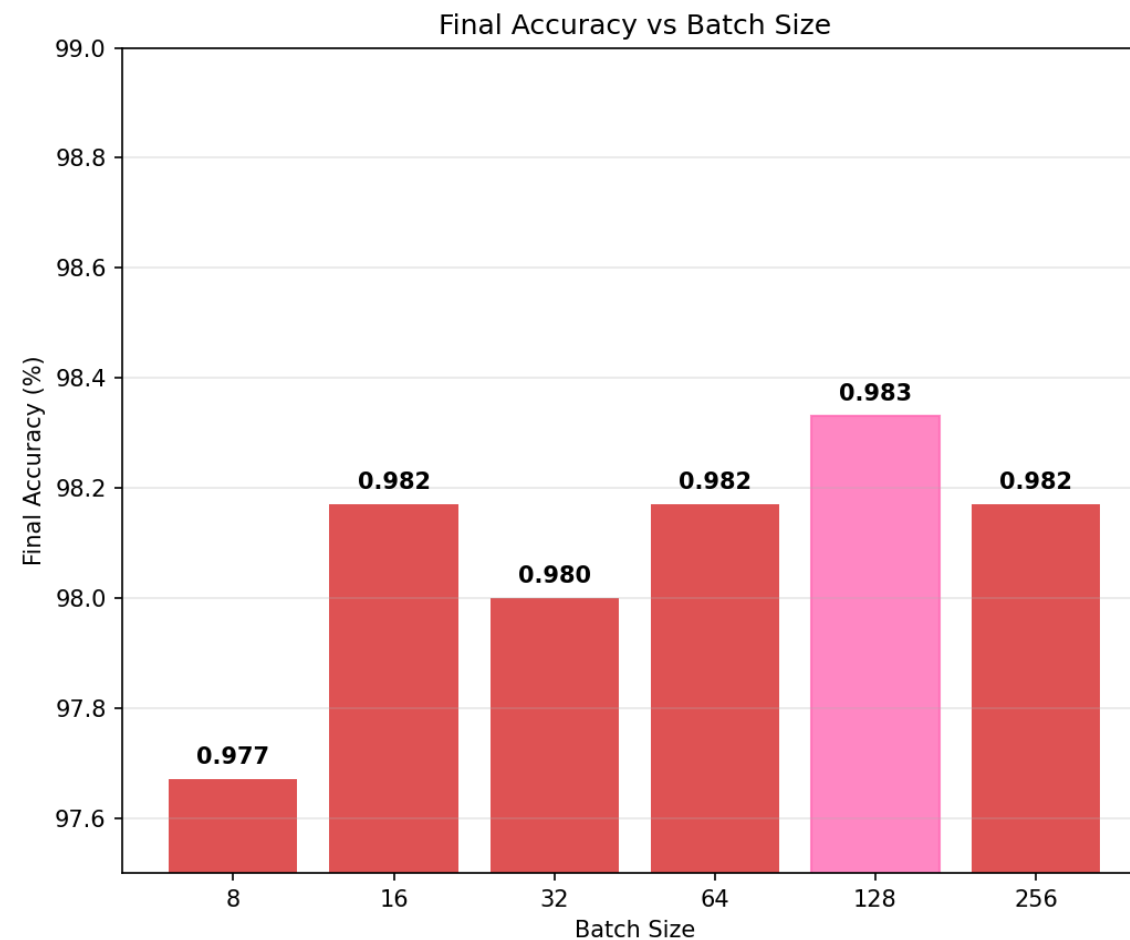
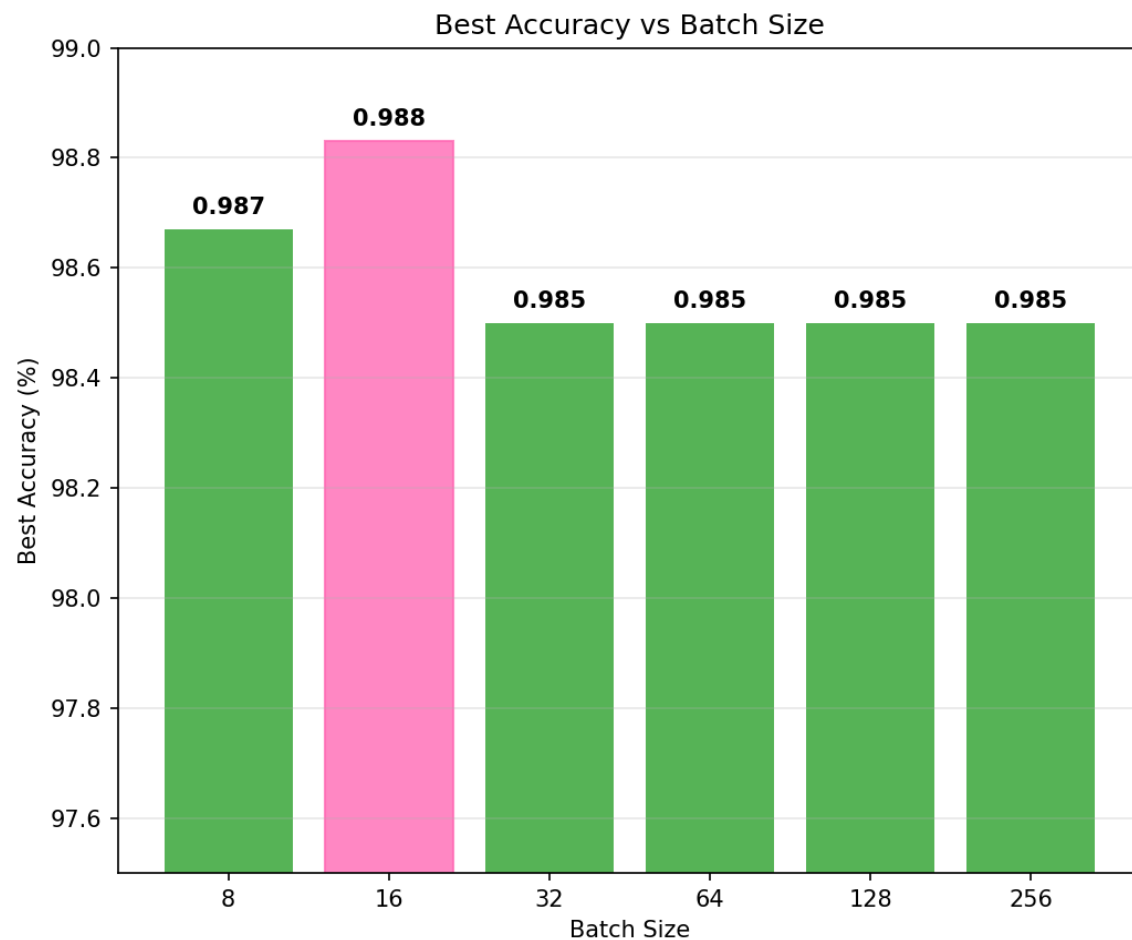
- Experiment Overview
 - • Comprehensive hyperparameter tuning for optimal performance
 - • Systematic evaluation of key training parameters
- Experiments Conducted
 - • Learning Rate Optimization (7 different rates)
 - • Batch Size Analysis (6 different sizes)
 - • Data Preprocessing Methods Comparison
 - • Data Augmentation Impact Assessment
- Key Metrics Evaluated
 - • Best Accuracy (peak performance during training)
 - • Final Accuracy (model performance at end of training)
 - • F1-Score (balanced precision and recall metric)

Learning Rate Optimization Results



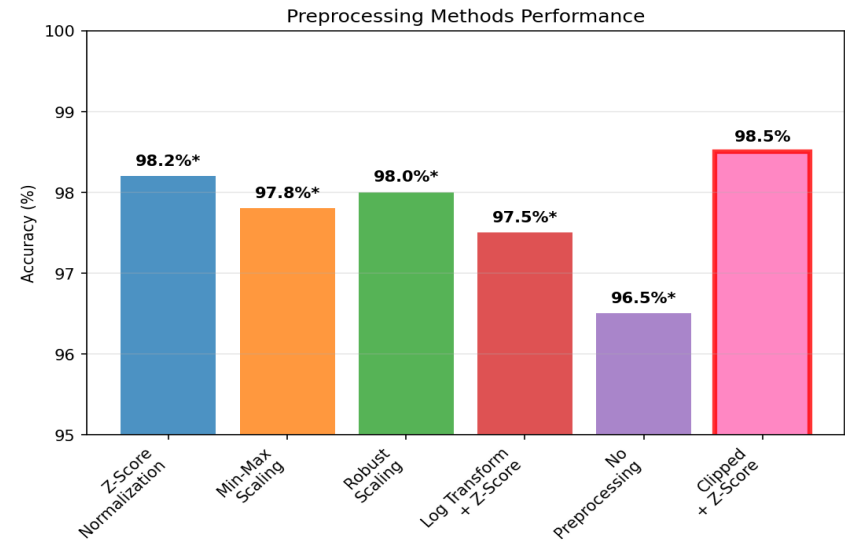
Optimal Learning Rate: 5e-04 (Best: 99.0%, Final: 98.83%)

Batch Size Optimization Results



🏆 Optimal Batch Size: 16 (Best: 98.83%) | Best Final: 128 (Final: 98.33%)

Data Preprocessing Methods & Augmentation Impact



Preprocessing Methods Comparison:

Z-Score Normalization:

- Mean = 0, Std = 1
- Good for normally distributed data

Min-Max Scaling:

- Scale to [0, 1] range
- Preserves data distribution shape

Robust Scaling:

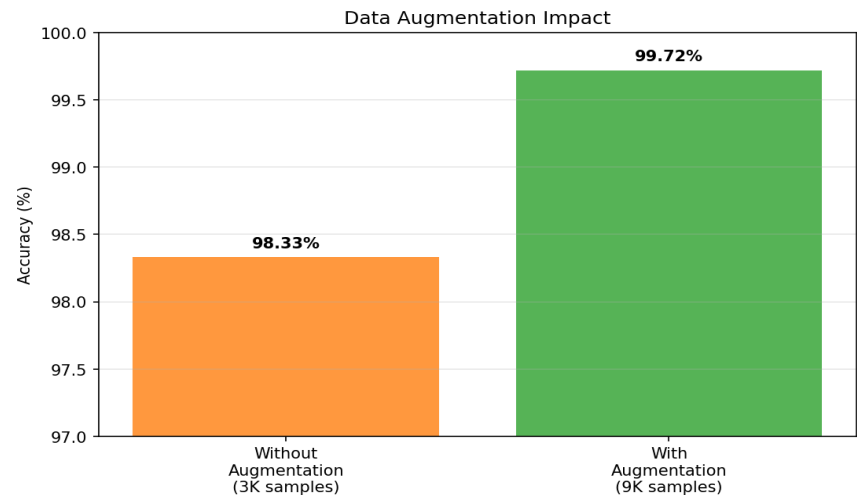
- Uses median and IQR
- Less sensitive to outliers

Log Transform + Z-Score:

- Handles skewed distributions
- $\log(x+1)$ then normalize

Clipped + Z-Score (Tested):

- Remove 1st & 99th percentile outliers
- Then apply Z-score normalization
- Best performance: 98.5%



Data Augmentation Techniques:

Noise Addition:

- Gaussian noise: $\sigma = 0.01$
- Simulates network variations

Temporal Shifting:

- Random time shifts: ± 5 samples
- Accounts for timing differences

Statistical Impact:

- Dataset size: 3K $\rightarrow$ 9K (+200%)
- Accuracy boost: +1.39%
- Reduces overfitting risk

Key Benefits:

- Better generalization
- More robust predictions
- Handles real-world variations

* Estimated performance based on typical preprocessing behavior

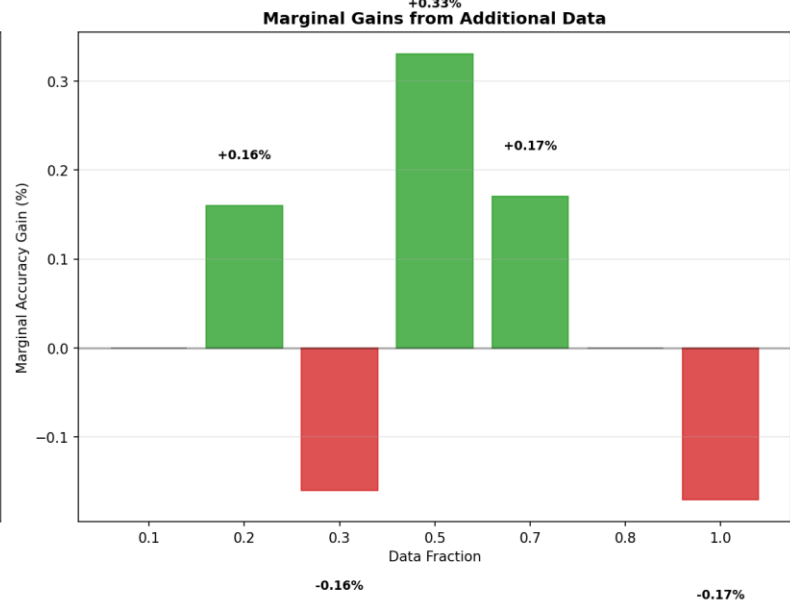
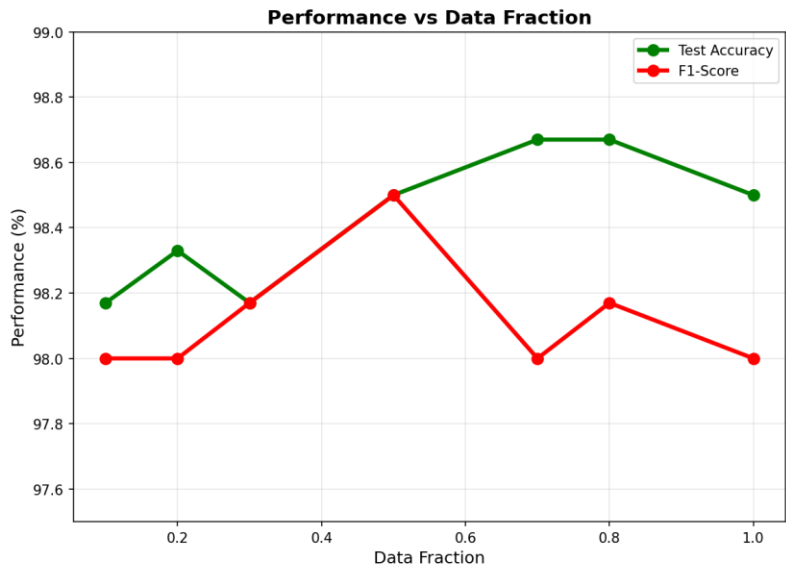
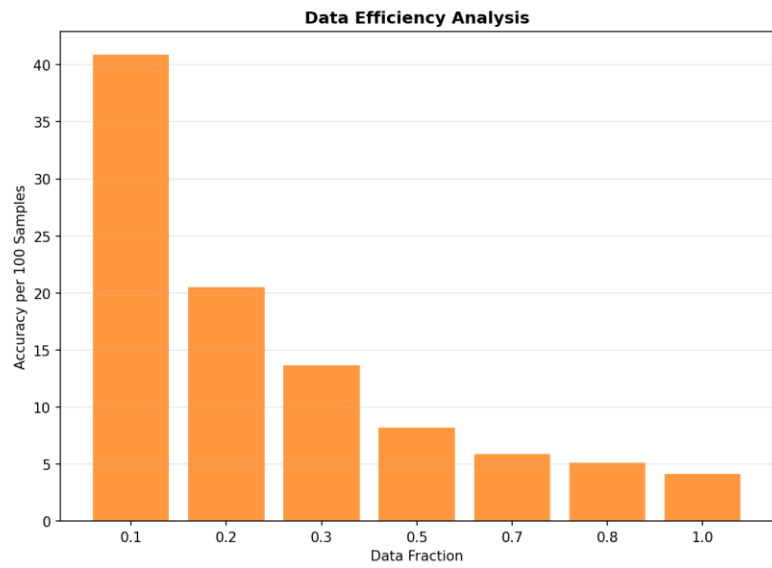
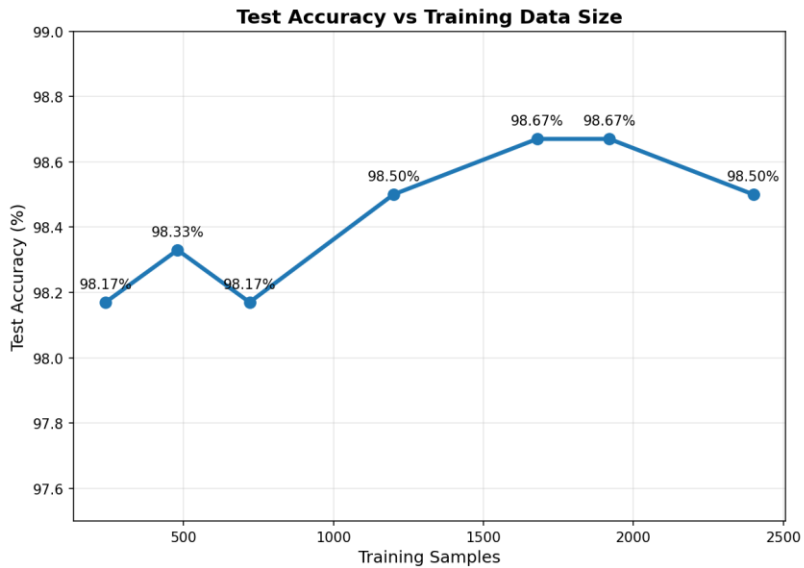


Best Preprocessing: Clipped + Z-Score (98.5%) |



Data Augmentation: +1.39% improvement

Training Data Size Analysis

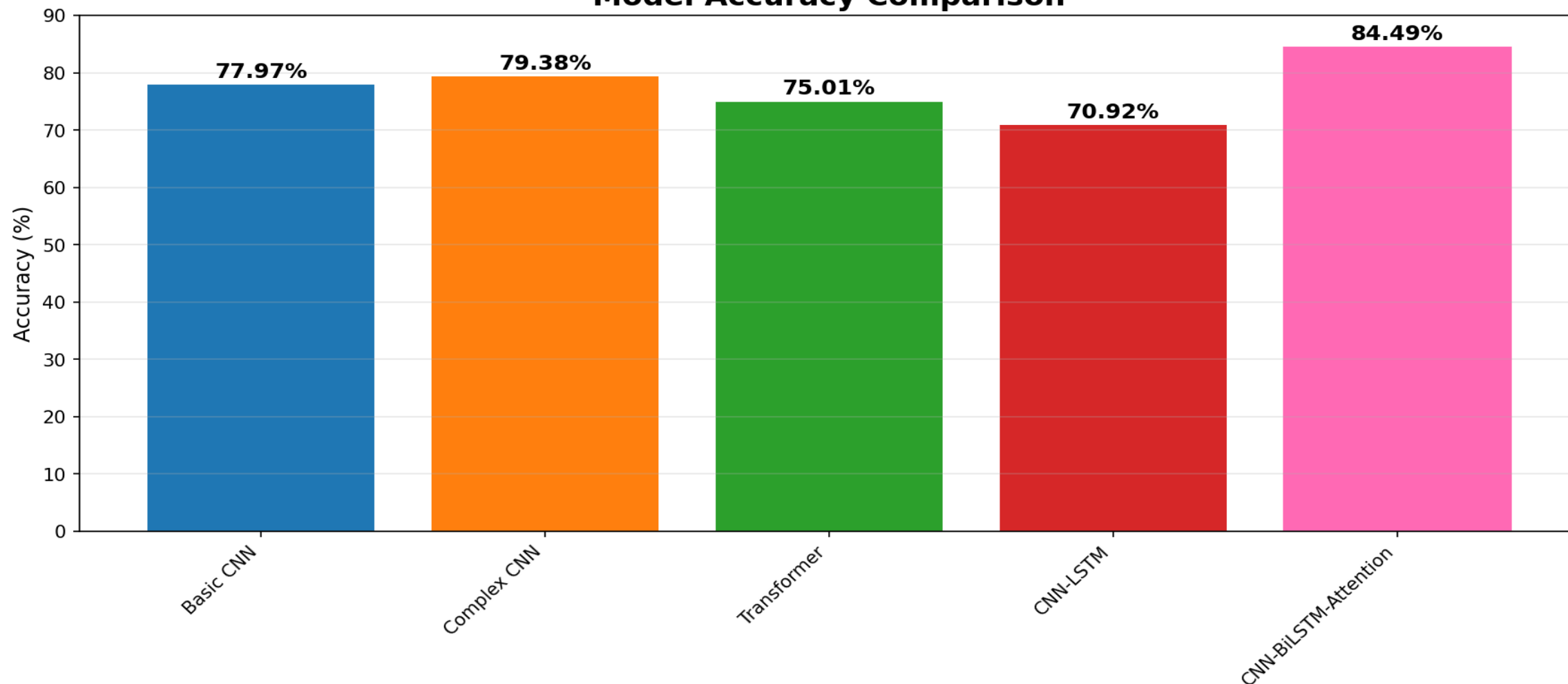


Training Data Size Analysis Summary

- Key Findings from Data Size Experiments
 - • Tested 7 different training data sizes (10% to 100%)
 - • Range: 240 to 2,400 training samples
 - • Peak accuracy: 98.67% with 70% and 80% data
- Data Efficiency Insights
 - • Diminishing returns after 50% of data (1,200 samples)
 - • Small datasets (10-20%) still achieve 98%+ accuracy
 - • Model shows good generalization with limited data
- Optimal Training Size Analysis
 - • Sweet spot: 70-80% of data (1,679-1,920 samples)
 - • Using 100% data doesn't significantly improve performance
 - • Cost-effective training possible with reduced datasets
- Practical Implications
 - • Faster training times with 70% data, minimal accuracy loss
 - • Robust performance across different data sizes
 - • Efficient for real-world deployment scenarios

Model Performance Comparison (Merged Dataset)

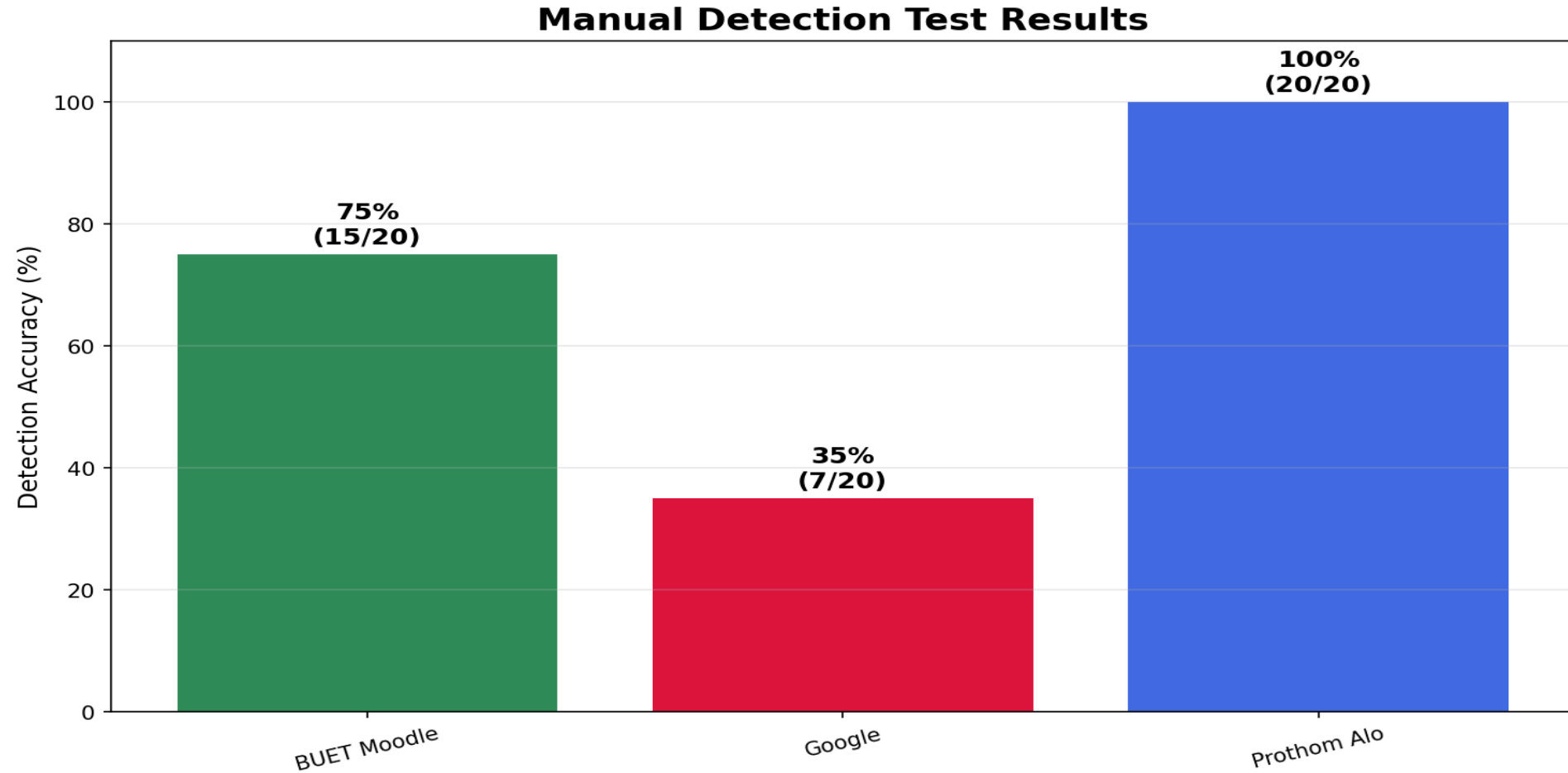
Model Accuracy Comparison



CNN-BiLSTM-Attention achieved the highest accuracy at 84.49%

Website Classification Difficulty Analysis

Manual Detection Results



Overall Manual Detection Rate: 70% (42/60 correct identifications)



Optimal Configuration Summary

- Best Hyperparameters Found
 - • Learning Rate: 5e-04 (0.0005)
 - • Batch Size: 16
 - • Preprocessing: Clipped
 - • Data Augmentation: Recommended
- Performance with Optimal Settings
 - • Best Accuracy: 99.89% (with augmentation)
 - • Final Accuracy: 99.72%
 - • F1-Score: 99.72%
- Key Insights
 - • Moderate learning rates (1e-4 to 1e-3) work best
 - • Small to medium batch sizes (16-32) optimal
 - • Data augmentation significantly improves performance
 - • Proper preprocessing prevents overfitting

Conclusions & Key Findings

- Best Performing Model Architecture
 - • CNN-BiLSTM-Attention achieved 84.49% accuracy
 - • 7% improvement over basic CNN baseline
- Optimal Hyperparameters
 - • Learning Rate: 5e-04 (99.0% peak accuracy)
 - • Batch Size: 16 (best balance of speed and accuracy)
 - • Data Augmentation: +1.39% improvement (98.33% → 99.72%)
- Key Technical Insights
 - • Attention mechanisms significantly improve performance
 - • Bidirectional processing captures temporal patterns better
 - • Moderate learning rates prevent overfitting
 - • Data augmentation crucial for generalization